

Banco de prompts + templates claude.md y design.md

Los prompts exactos de los dos casos de la masterclass, listos para copiar y pegar, más las plantillas de los dos archivos mágicos.

Cómo usar este banco

Los prompts siguen siempre el mismo orden: **1) comprender** → **2) construir** → **3) refinar** → **4) validar**. No pidas código en el primer prompt: primero obliga a la IA a leer tu contexto. Así dirige un director creativo.

Caso 1 · Límite de fertilidad por ciclos

Prompt 1.1 — Comprensión del contexto

```
Lee claude.md y design.md. Eres un Product Designer nuevo en Collective Health. El brief: "agregar soporte para límites por ciclo de fertilidad, máximo 3 ciclos de por vida, siguiendo el patrón Seleccionar límite → Tipo de límite → Aplicar límite a". Dame:
1. Un resumen del problema en 3 bullets.
2. Las pantallas/componentes que hay que tocar.
3. Los riesgos UX que detectas (confusión ciclo/cantidad, etc.).
4. 3 preguntas que le harías al PM antes de diseñar.
```

Prompt 1.2 — Generación del componente

```
Crea un componente <LimitConfigurationFlow /> en app/components/:
- 3 pasos: Step 1 "Seleccionar límite", Step 2 "Tipo de límite", Step 3 "Aplicar límite a".
- Step 2: tarjetas seleccionables "Monto", "Cantidad" y "Ciclo" (esta última con badge NUEVO).
- Step 3 con "Ciclo": input numérico, label "Máximo de ciclos de por vida", helper text "Recomendado: 3 ciclos".
- Usa SOLO componentes y tokens definidos en design.md.
- Estado con useState, sin librerías externas.
- Cada step con título, descripción y botones Atrás/Siguiente.
```

Prompt 1.3 — Microcopy y estados

```
Ahora agrega:
1. Estado de éxito: toast verde "Límite de ciclos guardado para cobertura de fertilidad".
```

2. Estado de error si ingresa 0 o negativo: "El máximo de ciclos debe ser mayor a 0".
3. Warning si ingresa más de 5: "Más de 5 ciclos requiere aprobación del cliente. ¿Confirmas igual?".
4. Microcopy del botón primario según tipo de límite elegido.

Prompt 1.4 — Validación heurística

Revisa `<LimitConfigurationFlow />` contra estas 5 heurísticas de Nielsen:

1. Visibilidad del estado del sistema (¿sabe en qué step está?)
2. Coincidencia sistema/mundo real (¿"ciclo" es claro?)
3. Prevención de errores (¿validamos antes de avanzar?)
4. Reconocimiento antes que recuerdo (¿opciones visibles?)
5. Diseño minimalista (¿hay ruido visual innecesario?)

Para cada una: ¿se cumple? ¿qué mejorar?

Caso 2 · Verificación de TIN de proveedores

Prompt 2.1 — Comprensión del problema

Lee `claude.md` y `design.md`. Vamos a diseñar la pantalla de verificación de TIN para Falcor Provider Suite.

Contexto:

- Collective Health paga a proveedores usando TINs de reclamos.
- El TIN se valida contra IRS y W-9.
- Hoy el proceso es manual y poco visible.
- Estados: Pass / Fail / Manual override / Vendor-check-pending.

Dame: mapa del flujo actual, mapa del flujo propuesto, qué ve el usuario en cada estado y qué acción puede tomar.

Prompt 2.2 — Pantalla principal

Crea `app/providers/page.tsx` con:

1. Barra de búsqueda por TIN o nombre de proveedor.
2. Tabla: Nombre, TIN, Estado, Acción.
3. Badges por estado: Pass (verde) / Fail (rojo) / Manual override (amarillo) / Vendor-check-pending (gris).
4. Mock data: 6 proveedores, al menos uno de cada estado.
5. Filtros por estado sobre la tabla.
6. Paginación simple (10 por página).

Prompt 2.3 — Acción de manual override

Agrega a la tabla:

1. Botón "Marcar como verificado" en filas Fail o Vendor-check-pending.
2. Modal de confirmación:
 - Título: "Confirmar verificación manual"
 - Body: "¿Confirmas que el TIN {TIN} de {Nombre} está validado? Esta acción quedará registrada con tu usuario."
 - Campo obligatorio "Motivo" (textarea, mínimo 20 caracteres).
 - Botones: Cancelar / Confirmar verificación.
3. Al confirmar: estado pasa a "Manual override" + toast de éxito.
4. Botón deshabilitado en filas Pass o Manual override.

Prompt 2.4 — Microcopy operacional

Propón microcopy para:

1. Empty state sin resultados: "No encontramos proveedores con ese criterio. Prueba con otro TIN o nombre."
2. Loading: "Buscando proveedores..."
3. Helper del buscador: "Ingresa el TIN (9 dígitos) o parte del nombre del proveedor."

4. Confirmación post-override: "TIN {TIN} marcado como verificado por {Usuario} el {Fecha}."
5. Tooltip de Vendor-check-pending y error de motivo corto.

Template · claude.md (el brief persistente)

Copia esto en un archivo `claude.md` en la raíz de tu proyecto y adapta los valores entre `{{ }}`.

```
# Project Brief – {{Nombre del Proyecto}}
## Contexto del proyecto
**Producto:** {{ej: Falcor Provider Suite}}
**Usuario principal:** {{ej: Provider Operations Staff}}
**Problema que resuelve:** {{1-2 oraciones}}

## Stack técnico
- Framework: Next.js 14+ (App Router) · TypeScript
- Estilos: Tailwind CSS · Componentes: shadcn/ui o custom
- Estado: React useState · Icons: lucide-react

## Convenciones de código
- Componentes: PascalCase · Utilidades: camelCase
- Rutas: kebab-case · Constantes: UPPER_SNAKE
- TODO componente consume tokens de design.md (nada hardcodeado)
- NUNCA instalar librerías nuevas sin confirmación

## Restricciones y reglas de negocio
1. {{regla 1, ej: máx. 3 ciclos de por vida}}
2. {{regla 2, ej: 4 estados posibles de TIN}}
3. {{quién puede hacer qué, auditoría, etc.}}

## Definición de "Done" para una pantalla
- [ ] Respetar tokens de design.md (sin hardcodeos)
- [ ] Estados: loading, empty, success y error
- [ ] Accesible por teclado (Tab, Enter, Esc)
- [ ] Responsive · [ ] Commit con mensaje descriptivo

## Ejemplos de referencia
- {{links a componentes existentes que sirven de guía}}
```

Template · design.md (el design system vivo)

La única fuente de verdad para tokens visuales. La IA DEBE consumir estos tokens y NUNCA hardcodear valores.

```
# Design System – {{Nombre del Proyecto}}
## Color tokens
--color-primary-500: #3B82F6; /* default */
--color-primary-600: #2563EB; /* hover */
--color-success: #10B981; /* Pass */
--color-warning: #F59E0B; /* Manual override */
--color-danger: #EF4444; /* Fail */
--color-pending: #6B7280; /* Vendor-check-pending */

## Tipografía
--font-sans: 'Inter', system-ui, sans-serif;
--font-size-base: 16px; --font-size-sm: 14px;

## Spacing (escala 4px) y radii
--space-2: 8px; --space-4: 16px; --space-8: 32px;
```

```
--radius-md: 6px; --radius-lg: 8px;
## Componentes permitidos (lista blanca)
Button · Input · Select · Card · Badge · Table · Modal · Toast
(Si la IA propone otro componente, debe pedir permiso primero.)
## Reglas de uso
1. Un solo CTA primario por pantalla.
2. Estados de validación SIEMPRE con el Badge correspondiente.
3. Modales: Cancelar a la izquierda, Confirmar a la derecha.
4. Microcopy de error constructiva, nunca culpar al usuario.
## Prohibiciones absolutas
- NO hardcodear hexadecimales · NO usar !important
- NO crear componentes fuera de la lista blanca
```

Tip: Prueba de fuego: cambia `--color-primary-500` en `design.md` y pídele a Claude que actualice los componentes. Si todo se propaga solo, tu design system está vivo.